UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

EBB 320

Control Systems

Practical 2: Report

Group 28

Name and Surname	Student Number	Signature	% Contribution
A. van Niekerk	19007583		33
TS Fogwill	18321692		33
KC. Mattheus	17049271		33

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

October 12, 2021

Contents

List of Figures

1 Work Breakdown

1.1 A van Niekerk

- Introduction
- Motor Aim
- Motor Design
- Motor Simulation
- Motor Construction
- Motor Testing
- Motor Results
- Transfer Function (Task 5)
- Conclusion

1.2 TS Fogwill

- Line-sensor Aim
- Line-sensor Design
- Line-sensor Simulation
- Line-sensor Construction
- Line-sensor Testing
- Line-sensor Results
- Functional Block Diagram (Task 4)
- Conclusion

1.3 KC Mattheus

- State-control Aim
- State-control Design
- State-control Simulation
- State-control Construction
- State-control Testing
- State-control Results
- Linearized model (Task 6)
- Code flow-diagram (Task 7)
- Conclusion

2 Introduction and Aim

2.1 Introduction

The line-following robot is a basic concept that requires a number of complicated subsystems to operate and perform well. The subsystems consists of the motor and driver subsystem, the line sensor subsystem, and the state and control subsystem. The robot's movement and steering are controlled by the motor and drive system. The line sensor subsystem is in charge of tracking the location of the line under the sensor array. The state and control system is in charge of conveying information between the various systems and transitioning between various vehicle states.

The Autonomous Robotic Vehicle (ARV) is a hardware-implemented system that is designed to follow a specific coloured line by sensing the line and altering the motor speed accordingly, all while remaining undaunted. The system will contain an array of sensors for detecting the lines and converting the information to usable data, as well as a motor and a wheel for movement. The system must also be able to communicate through serial and cycle through multiple states while running on battery power.

2.2 Aim

Motor Subsystem

The aim for this subsystem is to design and build a motor driver circuit that will provide movement capabilities to the system as a whole. The motor systems need to work optimally and must be able to detect the wheel speed. The motor system will be controlled using a PWM signal that is generated on an Arduino Uno.

Furthermore the speed of the motor will be communicated to the micro controller through the use of serial communication. The micro controller will be able to send the wheel speed at any time when a specific control byte command is received. The circuit will have a decent design and to protect the circuitry, A optocoupler will be added to electrically isolate the motor from the rest of the circuit.

Line-Sensor Subsystem

The aim of the line-sensor subsystem is to design, simulate, build and test a line-sensor array that is capable of differentiating between green and white light. This allows the subsystem to know where a coloured line is relative to the sensor array (by detecting the coloured at a certain sensor in the array). The subsystem can use this information to extrapolate the deviation of the track from the central sensor and the direction in which this deviation occurs, which can then be used by the state-control subsystem to steer the ARV and follow the track.

The line-sensor subsystem communicates via serial to the state-control subsystem and measures data when prompted to do so (by the motor subsystem) via asynchronous, 8-bit serial communication.

State-control Subsystem

The aim of the state-control subsection in the practical is to design and build a state-control system that provides state capabilities to control the ARV as a whole. The state should be able to change

status via an activated touch button. The first state will do nothing until the button is pressed, and the second state receives, processes, and sends data.

In addition, the state-control subsystem should be able to communicate using serial. Serially received data includes hub connection, hub response, sensor line offset, and engine speed. Data sent serially includes current status, motor PWM, and motor RPM.

The circuit consists of one microprocessors, a serial connection and two capacitive contact surfaces.

3 Design Methodology

3.1 Motor Subsystem Design

In this section the motor subsystem design is shown. The subsystem consists of three major parts namely: motordriver, motor and encoder.

Table 1: Component list for Motor Subsystem

Component	**Component Type**	**Quantitiy**
IRF540N	Mosfet	1
4N25	Optocoupler	1
N20 Micro Metal Motor 100 RPM, 6V, Extended Shaft	Motor	1
N20 Magnetic Encoder	Encoder	1
1kOhm resistor	Resistor	2
10kOhm resistor	Resistor	1
Arduino Uno	Microprocessor	1
Jumper wires	Connectors	

Component information

Encoder:

The encoder sends a pulse each time the motor moves. The encoder that was used in the motor subsystems sends out 350 pulses per minute this is important for determining the RPM. The encoder should be soldered to the back of the DC motor. Below is a pin diagram of how it should be connected.

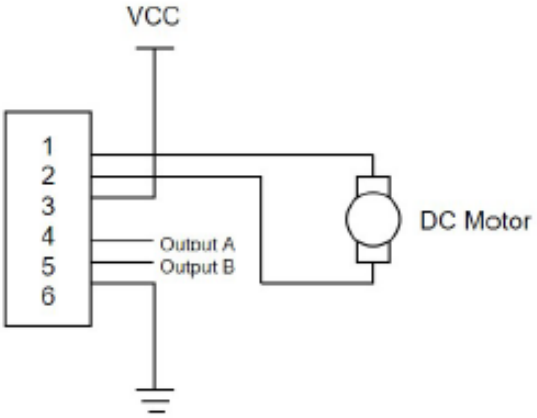

Figure 3.1.1: Encoder Pin map

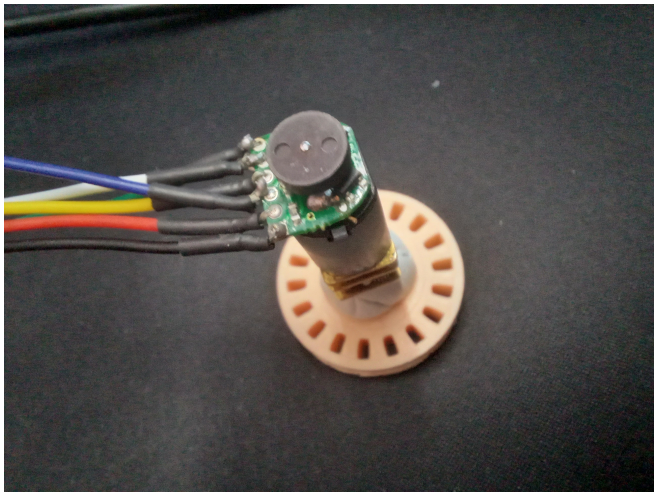

Figure 3.1.2: Encoder used in motor subsystem

MOSFET

For this design the IRF540N N-channel MOSFET was used. When the MOSFET is pulled high, it results in the maximum amount of current to flow through the device.

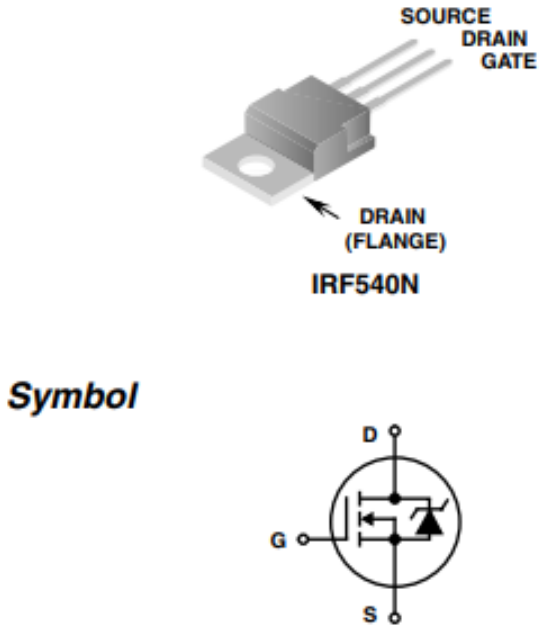

Figure 3.1.3: IRF540N information

Optocoupler

For this design the 4N25 optocoupler was used. The optocoupler is used to isolate the motor circuit from the arduino. The isolated circuit allows for the transfer of a higher voltage to the motor which results in higher RPM, without damaging the arduino.

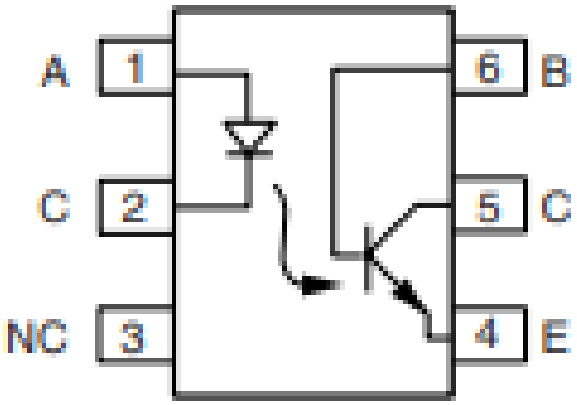

Figure 3.1.4: 4N25 Pin map

Motor Driver Circuit

A standard circuit was used for the construction of the motor driver. This circuit can be seen bellow as an actual implemented circuit and a simulation circuit that was used to test the functionality.

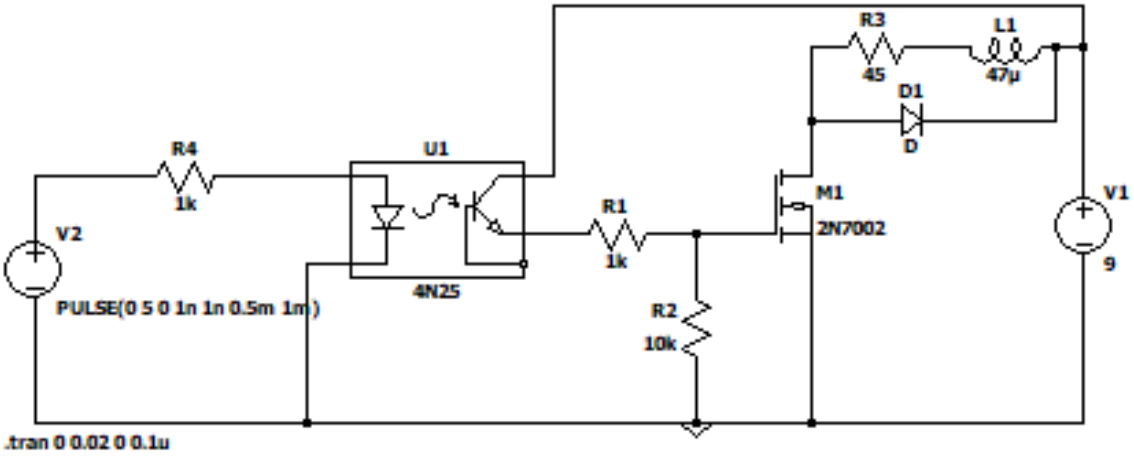

Figure 3.1.5: Motor Driver Circuit

In the simulations the 2N7002 MOSFET was used since it closely resembles the IRF540N.

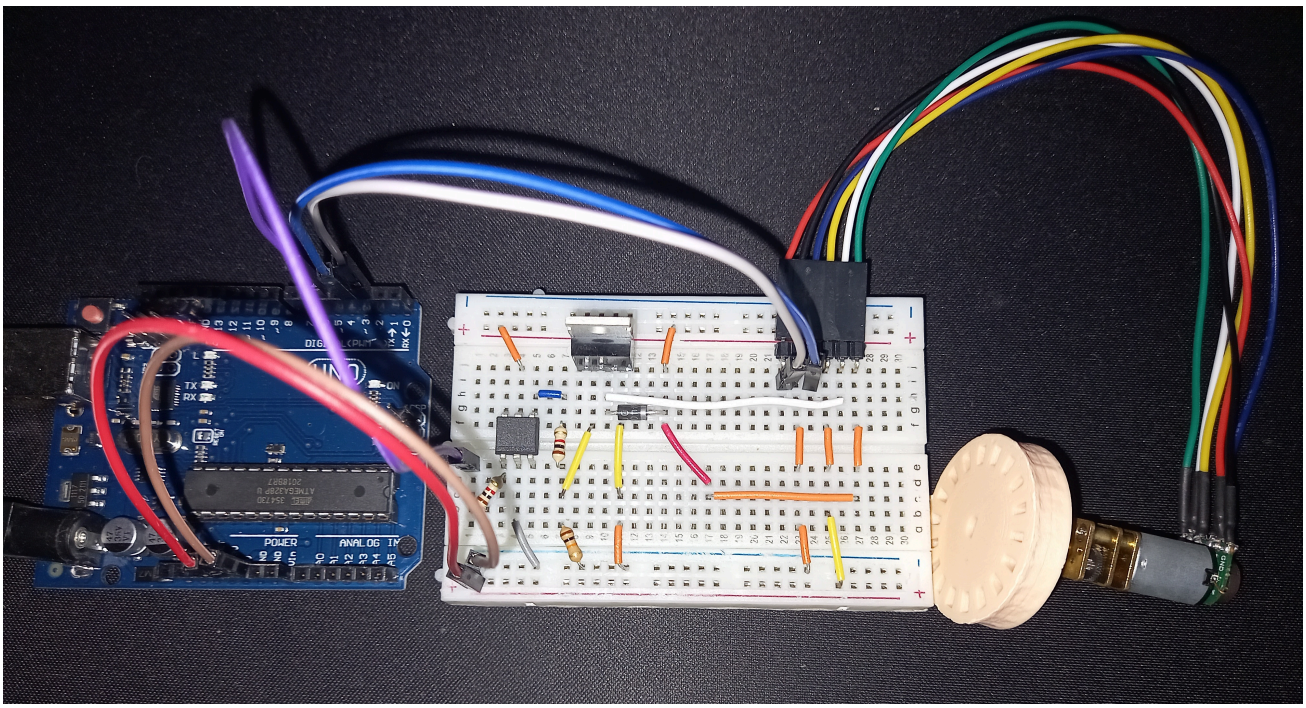

Figure 3.1.6: Built motor subsystem

Sensor to motor speed:

The RPM of the motor is measured by counting all the rising edges in one second times with 60. In the Arduino code there is a 200ms delay implemented to allow the pwm signal to be written to the pin this is accounted for by multiplying the pulse rate of the sensor, which is 350, by 1.2 that leads to a pulse rate of 420.

$$RPM = \frac{\text{Pulses in one second} \times 60}{420} \tag{1.1}$$

Input step-up for subsystem

To calculate the PWM range first we have to calculate the minimum PWM and the maximum PWM.

The minimum PWM signal was found by adding one to the PWM signal from 0 every 2 seconds until the motor starts to move. The minimum PWM signal was determined as 2.

To determine the Maximum PWM range the RPM speed was used. The PWm signal was incremented every 2 seconds if the previous RPM is less than the current RPM. It was concluded that the maximum rpm was reached at a PWM signal of 120.

The PWM range can thus be calculated as 118.

$$PWM_x = PWM_{min} + x/100 \times 118 \tag{1.2}$$

3.2 Line-sensor Subsystem Design

Components

The following components and equipment were used in the design and implementation of the line sensor subsystem.

1. Arduino Uno
2. 5 x SFH203 photodiodes
3. 5 x LM358P op-amps
4. 5 x 2.7MΩ resistors
5. 5 x 1kΩ resistors
6. 5 x 270Ω resistors
7. Jumper cables and single core wiring
8. Homemade housing unit
9. Multimeter

The housing unit was constructed using cardboard for a frame, reinforced with duct tape. Separators (to separate each sensor in the array) were made from cardboard and grooves were cut into the frame such that the separators can slot into the frame. Holes were then poked into the back of the frame to allow the legs of the photodiodes to stick through. These are then held in place with prestik.

Circuit Design

In order to detect colour a photo-detection circuit must first be built. The following figure as seen in figure 3.2.1, shows said photo-detection circuit.

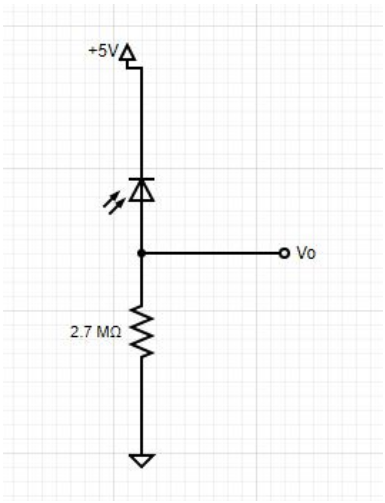

Figure 3.2.1: Photo-detection circuit diagram

This circuit detects light and converts said into electrical current and voltage. By comparing different voltage levels of different colour readings, different colours can be detected. However the voltage response is still relatively small. Therefore the output, V_o, must be amplified using a standard non-inverting amplifier as shown below in figure 3.2.2.

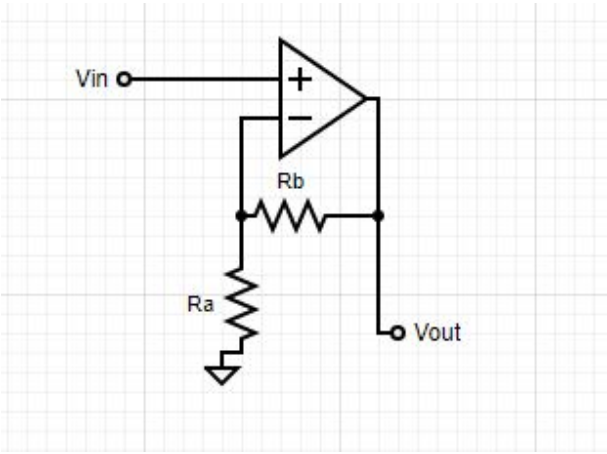

Figure 3.2.2: Amplifier circuit diagram

This amplifier amplifies the input voltage, V_{in}, by the following relationship:

$$V_{out} = A \times V_{in}$$
$$A = 1 + \frac{R_b}{R_a}$$

As can be seen in table 2, the pre-amplified values are small and in the range of [0.3, 0.7]. Therefore A is chosen as:

$$A \approx 6$$
$$5 = \frac{R_b}{R_a}$$

Choose $R_b = 1k\Omega$

$$\therefore R_a = \frac{1000}{5}$$
$$R_a = 200\Omega$$

Therefore choose $R_a = 270\Omega$

$$A = 1 + \frac{R_b}{R_a}$$
$$A = 4.7$$

To ensure that accurate line detection always occurs and to ensure that the state and control subsystem has as much accurate data to work with as possible, the sensor array will include 5 sensors in it. Therefore the full circuit of the line-sensor is shown in figure 3.2.3.

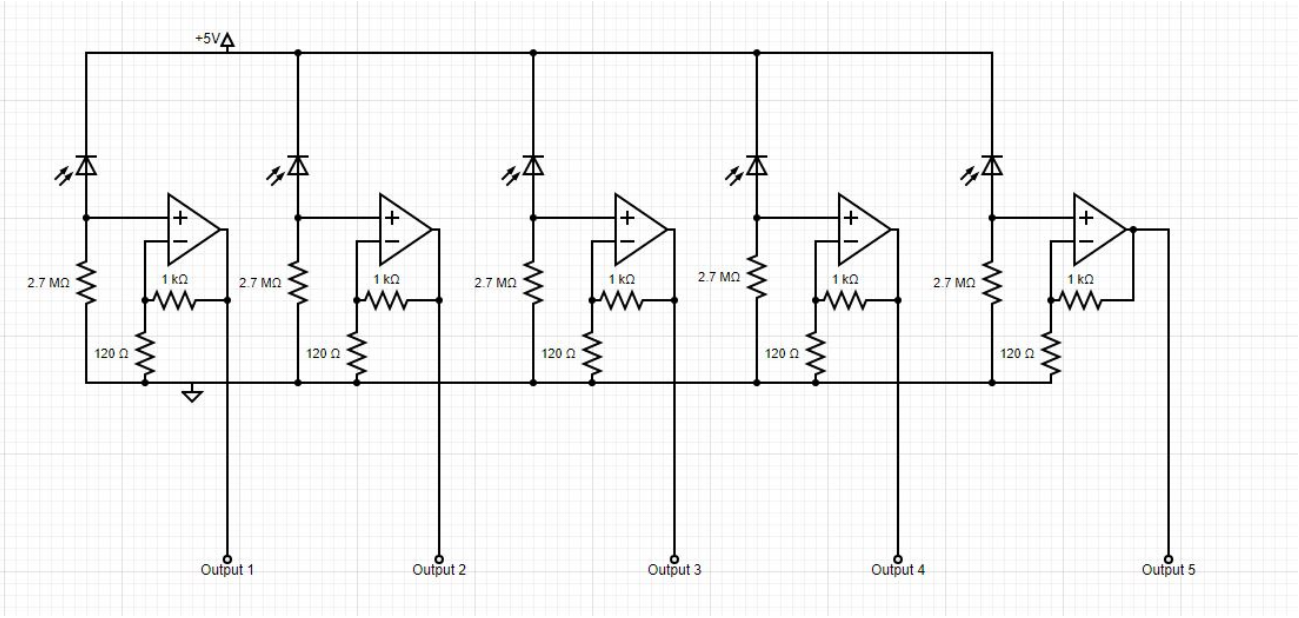

Figure 3.2.3: Entire line sensor circuit diagram

The outputs of the sensors are connected to the analog inputs of the Arduino Uno (pins A0 to A4), allowing the micro-controller to read and process the sensor data. The final system is shown below in figure 3.2.4.

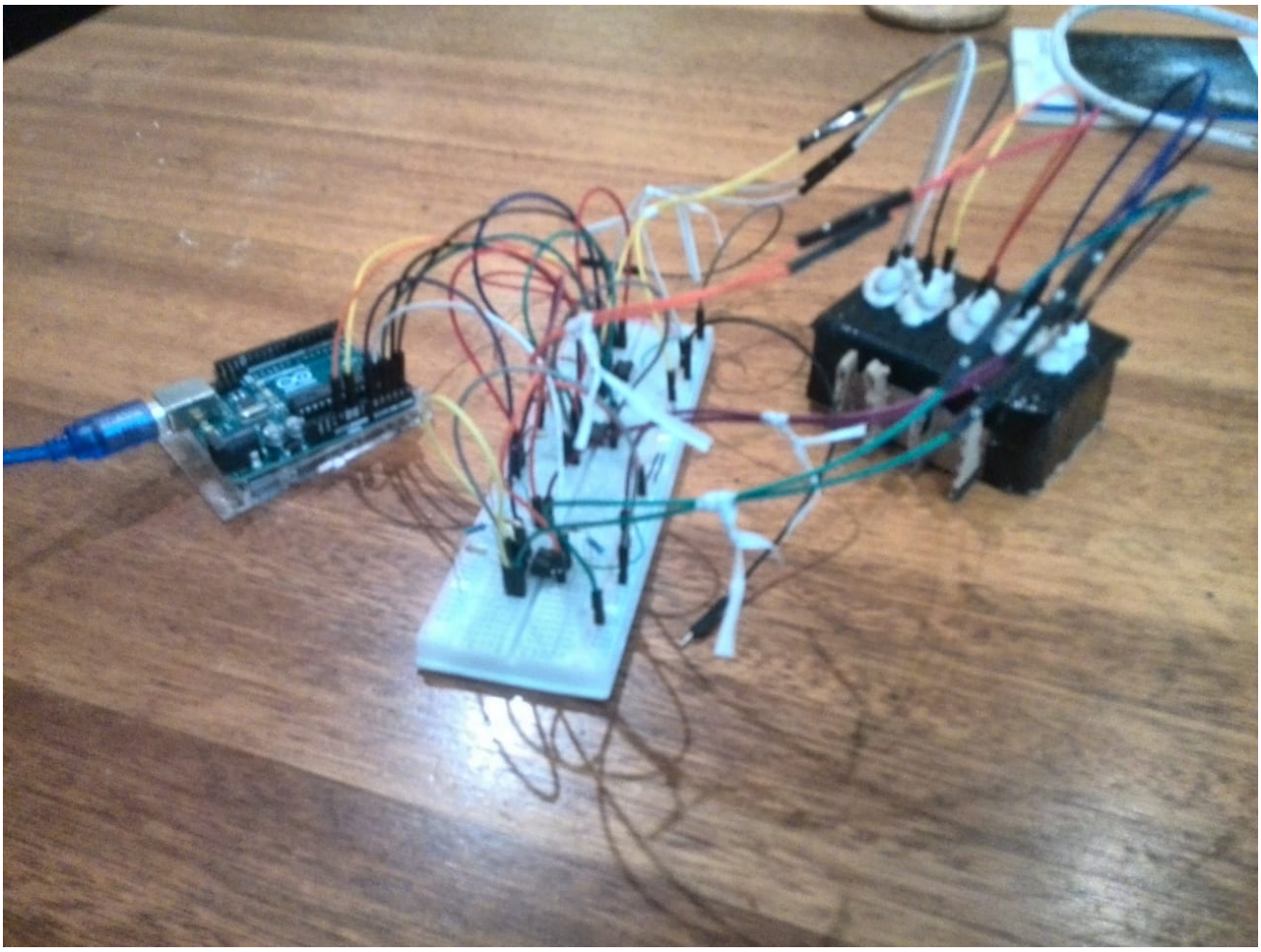

Figure 3.2.4: Line sensor subsystem image

3.3 State-control Subsystem Design

The state-control subsystem receive data from the other subsystems, display data for diagnostics, process the line offset, send the motor desired RPM and also to manage the system's state transitions. The main component of the subsystem will be the micro-processor, but this component is bought, thus most of this section will be focused on the designing on the software of the micro-processor. To display state, performance and diagnostic data a screen will be used as much more information can be displayed compared to an LED array. The Arduino IDE was used to program the micro-processor and Jupyter Lab was used to code and run the Serial logger.

One touch activated button was implemented to switch between states. The touch activated button was implemented by setting the associated digital pin to high and measuring the time it takes to go low. Because capacitance increases when a finger contacts the contact point, the pin takes longer to fall low. This variation in discharge duration is perceived as a touch-activated input, and the state is updated.

The following components were used to design and implement the SC3 subsystem:

- Arduino Nano micro-processor
- 1.3' OLED display

- A conducting coin
- Standard Jumper wires
- A breadboard
- One human finger

The following circuit diagram is used:

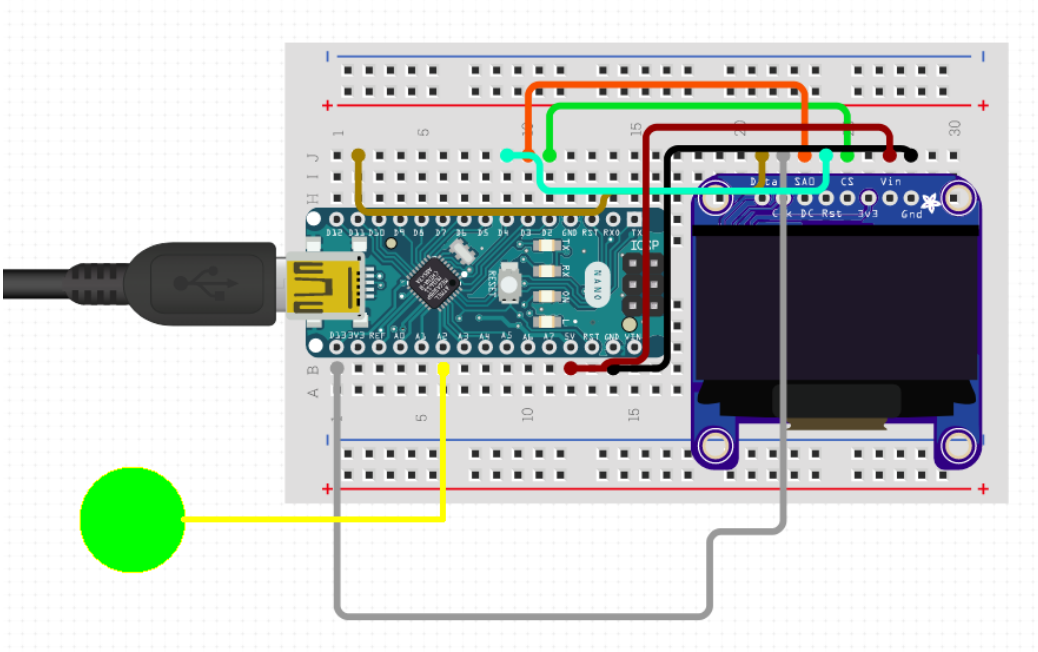

Figure 3.3.1: State-control subsystem circuit diagram

And built resulting in the following hardware realisation:

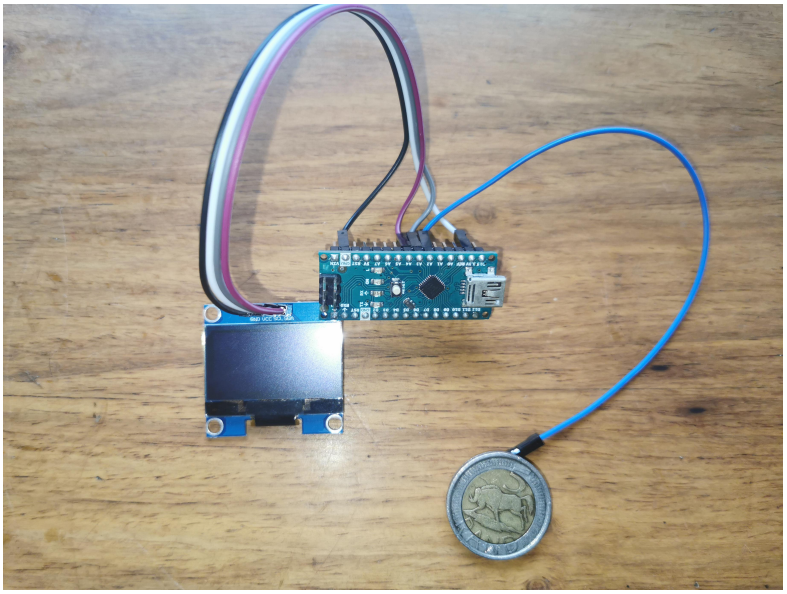

Figure 3.3.2: State-control subsystem circuit

4 Simulations

4.1 Motor Subsystem Simulations

The following circuit was used to simulate the motor driver circuit in LTspice to test the functionality of the design. The motor was replaced by a 45Ω resistor and 47 mH inductor. The MOSFET used in the simulations is a 2N7002 N-channel, which performs very similar to the IRF540N N-channel which will be used in the practical circuit.

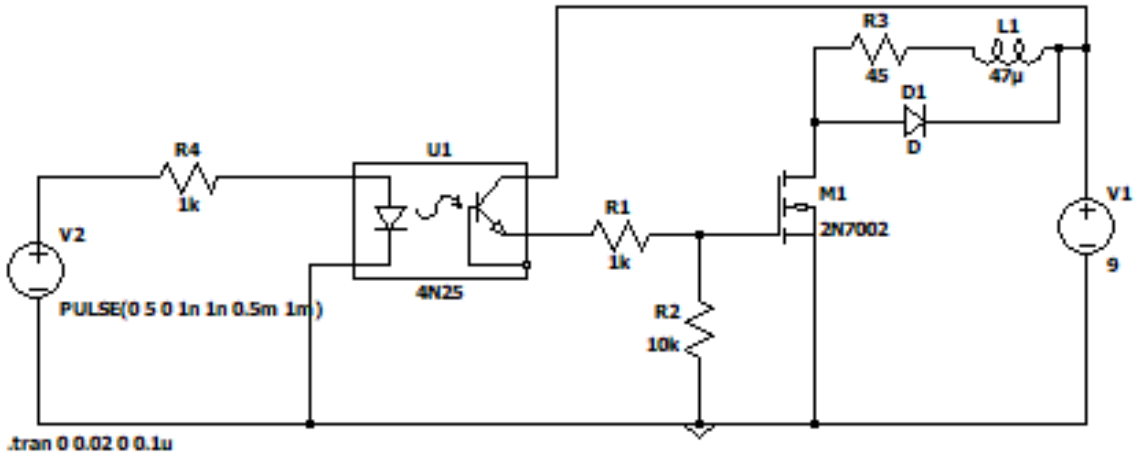

Figure 4.1.1: Motor Driver Circuit

A pulse signal with magnitude of 5V was generated at the input side to simulate the PWM from the Arduino Uno. The supplied voltage to the optocoupler is dampened by adding the 1kΩ in serie. This is done to protect the emitter of the optocoupler.

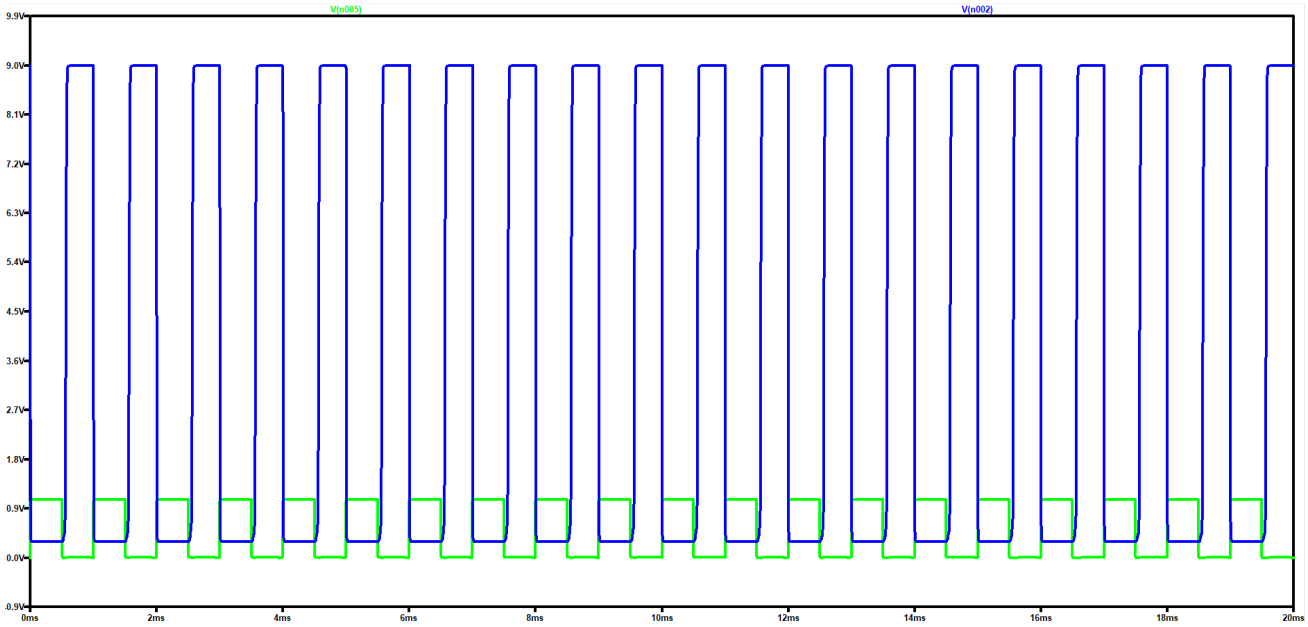

Figure 4.1.2: Simulation of Motor Driver Circuit

As seen in figure 4.1.2 the motor driver circuit successfully steps up to the maximum volatge provided with the falling endge of the pulse signal that represents the PWM of the Arduino.

4.2 Line-sensor Subsystem Simulations

In order to ensure that the amplifier circuit operates as needed, simulations were run. These simulations include the non-ideal and ideal amplifiers to fully understand the amplifiers behaviour.

Ideal Amplifier

Vin is set to a simple linear function with the following formula:

$$V_{in}(t) = t$$

This allows the simulation to show the amplification of the linear function.

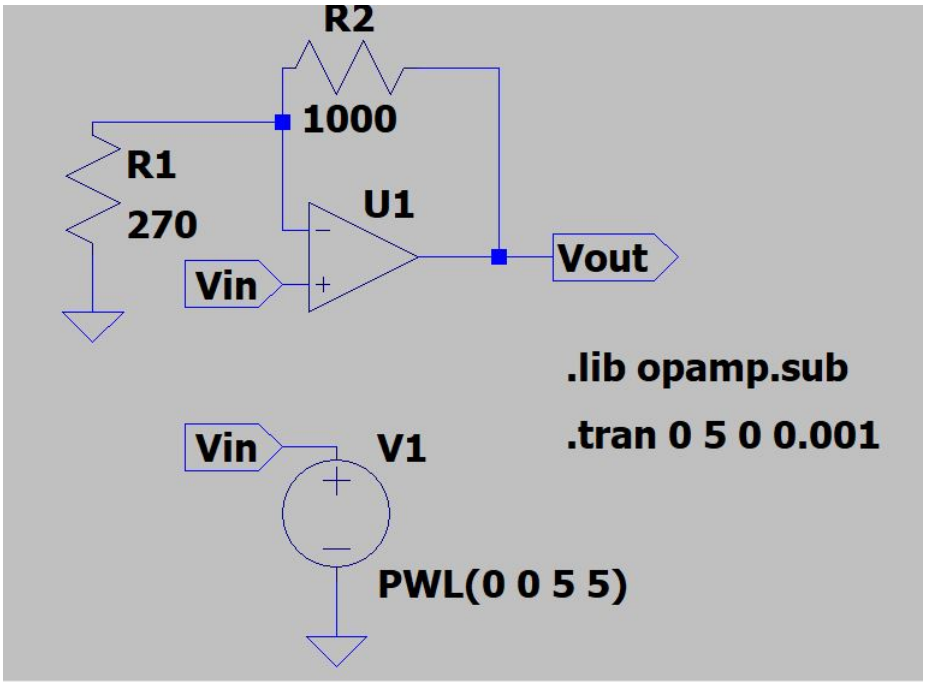

Figure 4.2.1: Line sensor ideal amplifier simulation

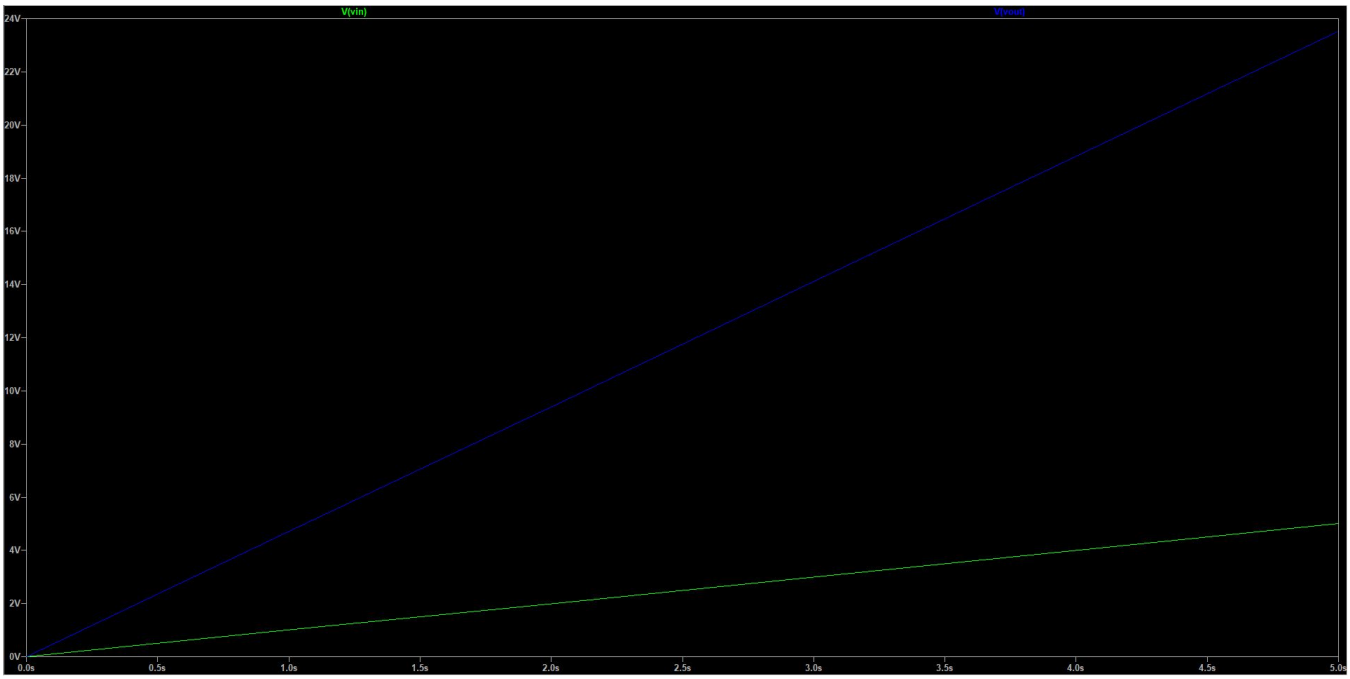

Figure 4.2.2: Line sensor ideal amplifier simulation results

Non-ideal Amplifier

As can been seen when viewing figure 2, the line sensor saturates at roughly 3.6V. Therefore the non-ideal amplifier simulations represents this by setting Vcc of the amplifier to 3.6V.

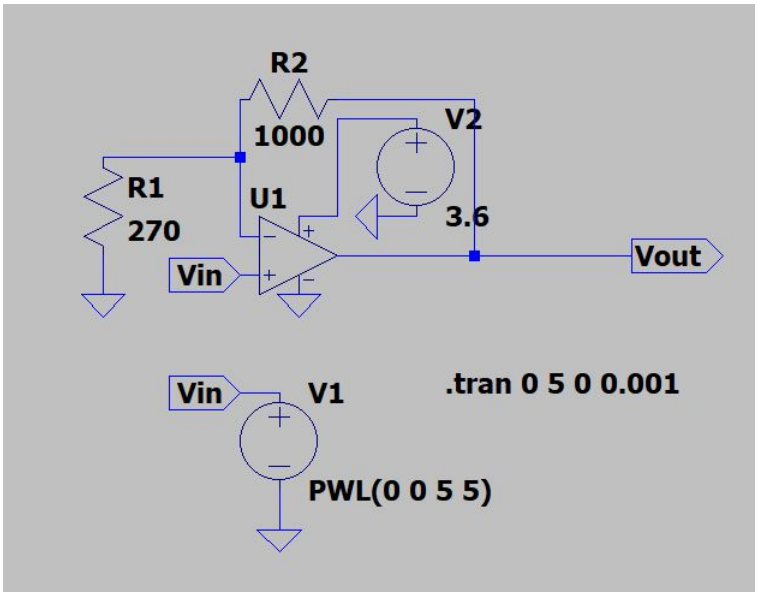

Figure 4.2.3: Line sensor non-ideal amplifier simulation

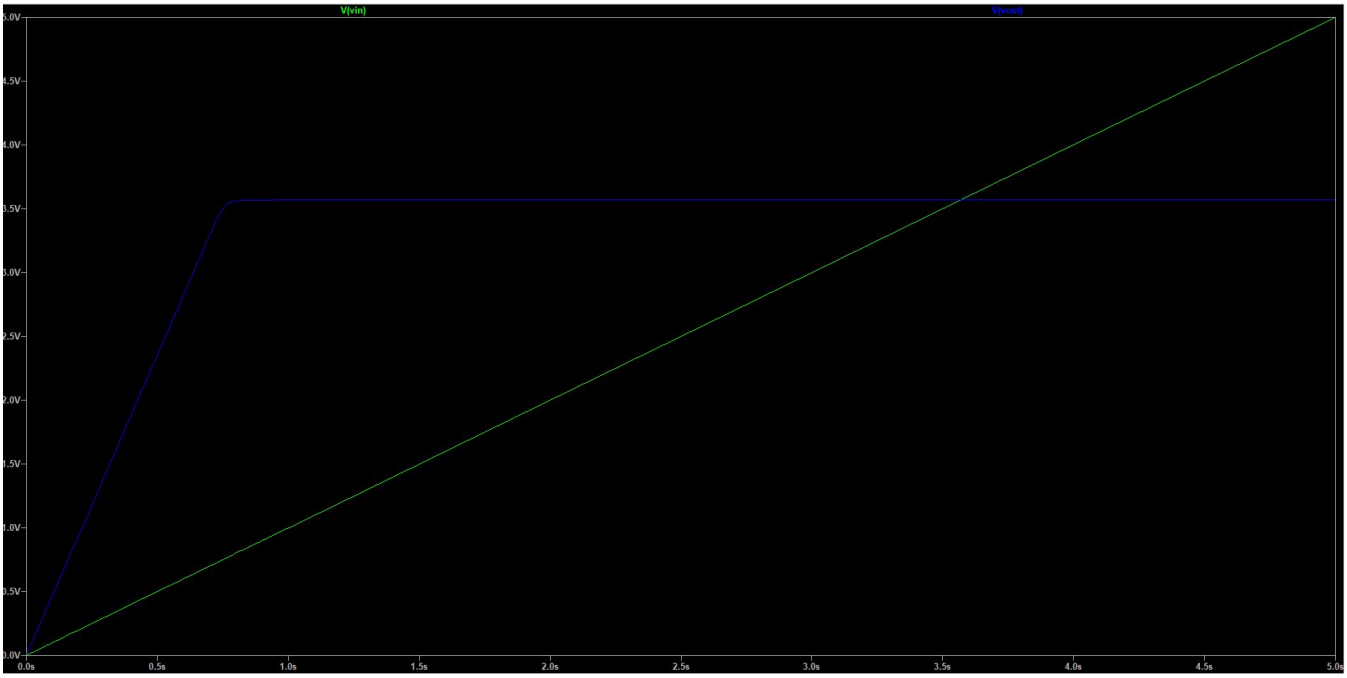

Figure 4.2.4: Line sensor non-ideal amplifier simulation results

4.3 State-control Subsystem Simulations

To simulate the subsystem a sinusoidal and then linear offset value was transmitted to the subsystem over a serial. The resulting speed of the left and right motor RPM, received over serial was saved using the Python logger.
The initial 90 offset values were from a sinusoidal wave with a simulated variation of ± 1 and maximum amplitude of $25mm$ and the second set of 60 offset values were linear between $0mm$ to $25mm$.
In practical 3 more advanced speed controllers will be implemented and tested but for this practical the following quadratic equation was implemented to calculate the motor PWM duty cycle percentage on the micro-controller:

$$MotorPWM = (\frac{\alpha^2}{1250} + 0.5) * 100 \tag{1.3}$$

Bellow is the graph showing the sent offset and resulting received emulated motor speeds:

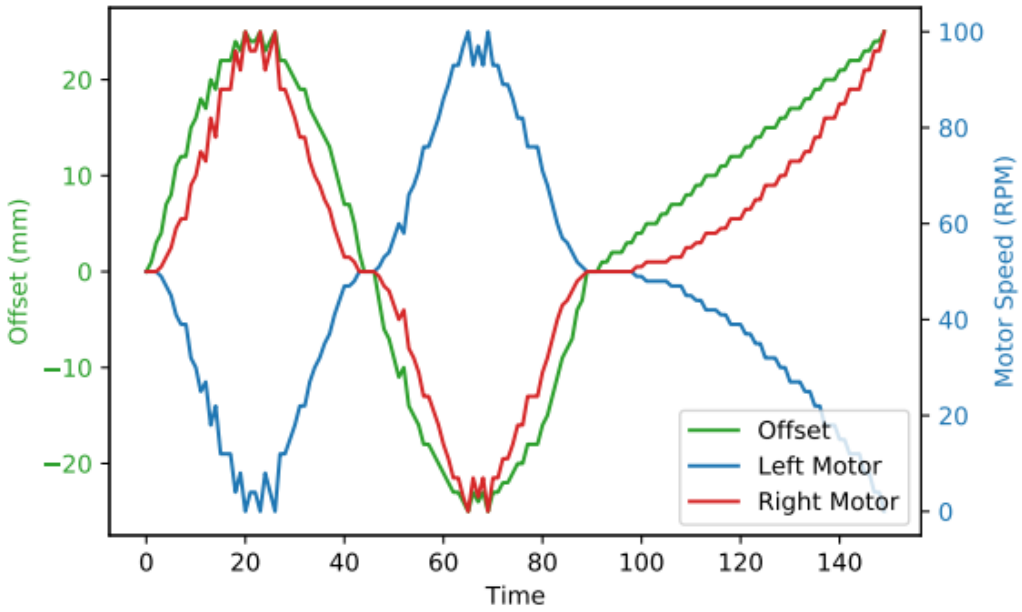

Figure 4.3.1: Graph showing the line offset and resulting motor speed

5 Results

5.1 Motor Subsystem's Results

Data Packets

The PWM signal of the motor is set through a serial command with a control byte of 161. When a packet with a control byte of 162 is sent the microchip needs to send back a packet of data with control byte of 163 followed by the current RPM.

```
Data packet sent to Arduino: [161,50,50,0]
Data packet recieved from Arduino:  b''
Data packet sent to Arduino: [162,100,0,0]
Data packet recieved from Arduino:  b'[163,118,12,0]\r\n'
Data packet sent to Arduino: [161,100,0,0]
Data packet recieved from Arduino:  b''
Data packet sent to Arduino: [162,100,0,0]
Data packet recieved from Arduino:  b'[163,129,1,0]\r\n'
Data packet sent to Arduino: [162,10,90,0]
Data packet recieved from Arduino:  b'[163,130,0,0]\r\n'
Data packet sent to Arduino: [161,10,90,0]
Data packet recieved from Arduino:  b''
Data packet sent to Arduino: [162,10,90,0]
Data packet recieved from Arduino:  b'[163,51,79,0]\r\n'
```

Figure 5.1.1: Packets sent to and recieved from Arduino Uno

As seen in figure 5.1.1 The Arduino successfully interprets and executes the commands of the sent data packets.

Signal step-up

The bellow figure will show the steped signal from 10% to 20% at 15 seconds.

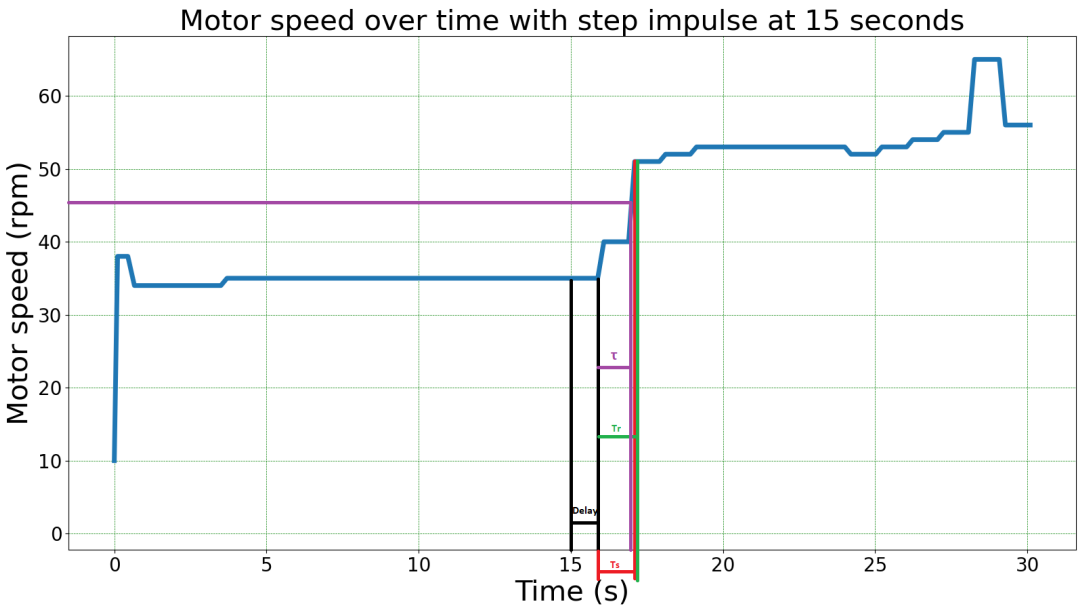

Figure 5.1.2: Input signal stepped from 10% to 20%

From figure 5.1.2 the following data can be obtained:

$$\begin{aligned} Delay &= 1s \\ T_s &= 1.46s \\ T_r &= 1.44s \\ \tau &= 1.38s \end{aligned}$$

5.2 Line-sensor Subsystem's Results

Sensor results

The voltage readings for green and white light, both pre- and post-amplification, are shown below in table 2.

Table 2: Sensor readings for the line sensor

Sensor No.	White (Amp)	Green (Amp)	White (pre-amp)	Green (pre-amp)
1	3.6V	1.55V	0.65V	0.31V
2	3.6V	1.5V	0.63V	0.30V
3	3.6V	1.6V	0.65V	0.33V
4	3.6V	1.6V	0.69V	0.31V
5	3.6V	1.7V	0.66V	0.33V

The data logging python algorithm as seen in appendix D, was run to simulate the operation of the

line sensor subsystem. A excerpt of the log is shown below in figure 5.2.1.

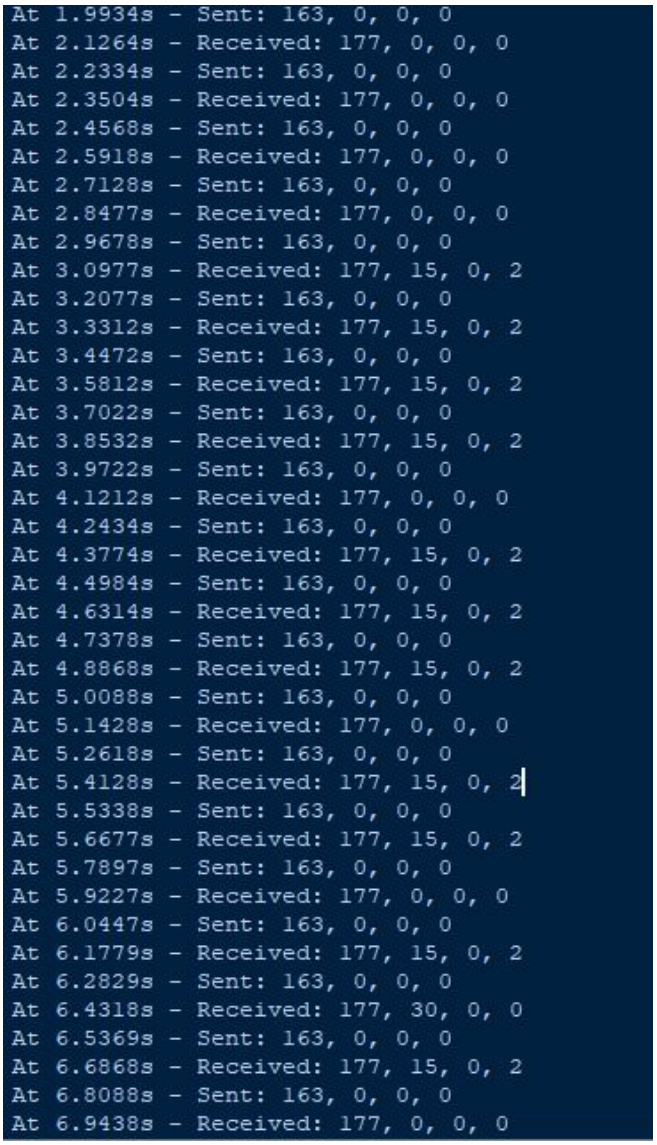

Figure 5.2.1: Data log excerpt of the line sensor subsystem

The data log is then written to a text file. This text file can then be read and the data within is plotted to figure 5.2.2.

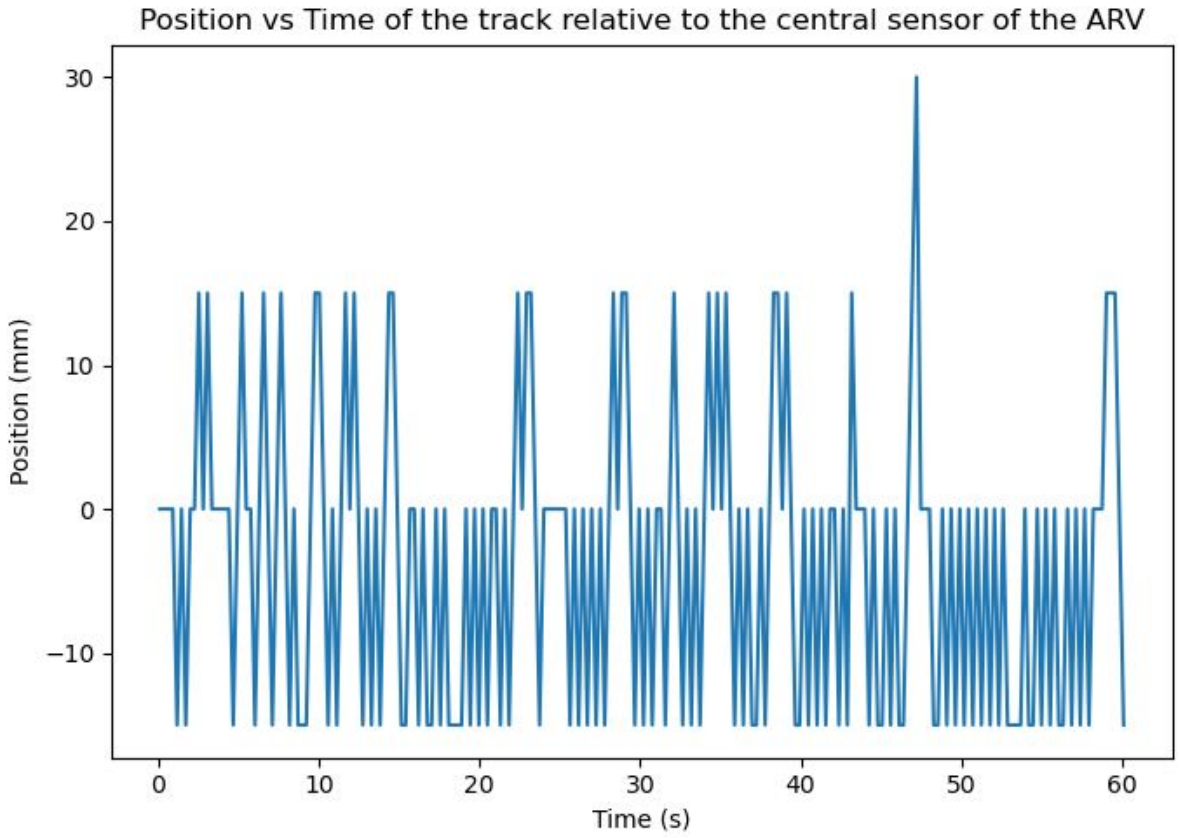

Figure 5.2.2: Position vs Time of sensor data

5.3 State-control Subsystem's Results

Simulating the motor control system with:

$$MotorPWM = (\frac{\alpha^2}{1250} + 0.5) * 100 \tag{1.4}$$

Resulted in the following motor speeds:

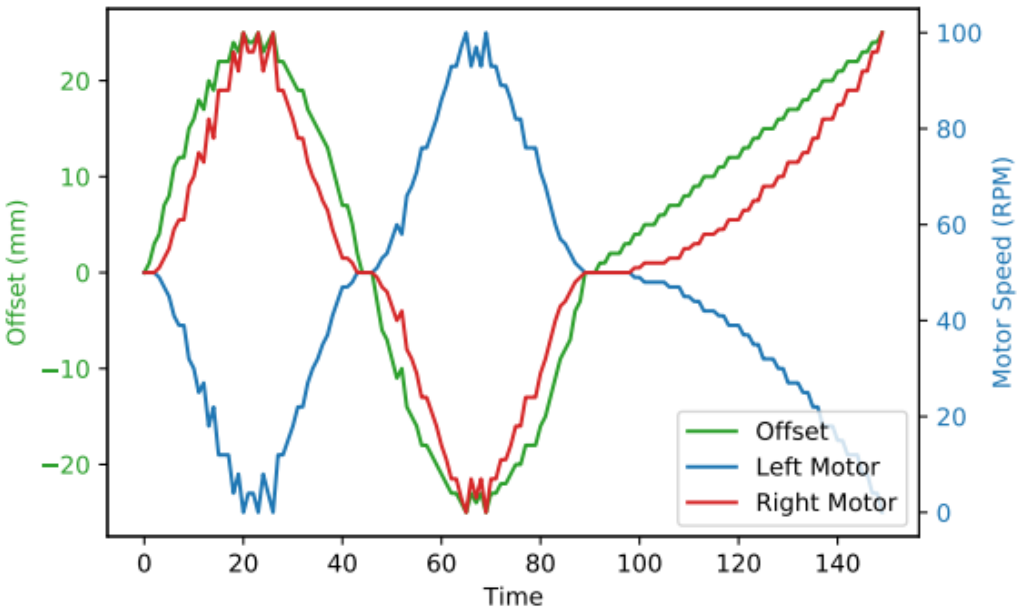

Figure 5.3.1: Graph showing the line offset and resulting motor speed

As seen in Figure 5.3.2, both motor speeds, with an offset of $0mm$, are at 50% of p_{max} ($p_{max} = 100RPM$) thus moving forward at half its maximum speed. As the offset increases (the ARV is to the right of the line), the right wheel starts to speed up and the left wheel slows down, moving it's nose to the left causing the line to re-centre. This difference in speed between the motors is exponential instead of linearly, thus the system reacts subtlety to smaller offsets and harshly to larger offsets. This is an attempt to reduce osculation in the ARV without a PID.

Below is an example of the screen output:

Figure 5.3.2: SC3 screen output

Below is an excerpt of the Python logger output:

```
Read: 0 0 0 0
Send: 16 0 0 0
Send: 16 1 0 0
Read: 177 0 0 0
Read: 163 50 50 0
Send: 145 50 50 0
Read: 177 5 0 0
Read: 163 50 50 0
Send: 145 52 48 0
```

```
Read: 177 10 0 0
Read: 163 51 49 0
Send: 145 58 42 0
Read: 177 15 0 0
Read: 163 55 45 0
Send: 145 68 32 0
Read: 177 20 0 0
Read: 163 61 39 0
Send: 145 82 18 0
```

6 Discussion

6.1 Motor Subsystem Discussion

The motor subsystem works and is operational, as evidenced by my results and simulations. This was accomplished by following the practical directions and putting the architecture figures provided to us into practice. The subsystem switches between PWM signals and can output encoder-derived RPM values. As shown in Figure 5.1.2 on page 18, the findings for the rise and settling time were likewise obtained through the subsystem. This figure was also acquired by using serial communication to log measurements from the encoder, plotting it, and obtaining the rise and settle times, both of which are very short when considering the load from the motor.

6.2 Line-sensor Subsystem Discussion

As can be seen when viewing 5.2.2, the line sensor has a tendency to output values of 15 and 0 more often than 30. This is an expected and intentional result. The algorithm prioritizes the sensors at the 2nd and 4th positions. This is because should more than 1 sensor pick up the track and the actual sensor is ambiguous, the sensor 15mm away are a safer choice. This ensures that the ARV moves in the correct direction but also allows it to turn more slowly to reduce the chances of getting lost. Additionally it can be seen that the there are more cases of 15 on the left than 15 on the right. Again this is due to algorithm checking the left sensor before then right sensors and if it discovers the track on one of the inner sensors, it uses that data without checking the others.

Table 2 shows the voltage readings for white and green, both pre- and post-amplification. The difference between these readings is very large. This ensures that the sensor can always differentiate between the two colours. Anything below 2.5V is seen as green and anything above 2.5V is seen as white.

The sensor subsystem is able to successfully and accurately detect the green track and then extrapolate the distance and direction between the track and the central sensor of the ARV when prompted. It is also able to effectively communicate this data to the state-control subsystem.

6.3 State-control Subsystem Discussion

The state change and serial communication worked successfully and as seen in Figure 5.3.2, it is observed that the speed of the right motor increases to its maximum values exponentially as the offset value increases as expected. Additionally the OLED screen successfully displays the state, offset and motor RPM.

6.4 Task 4: Functional Block Diagram

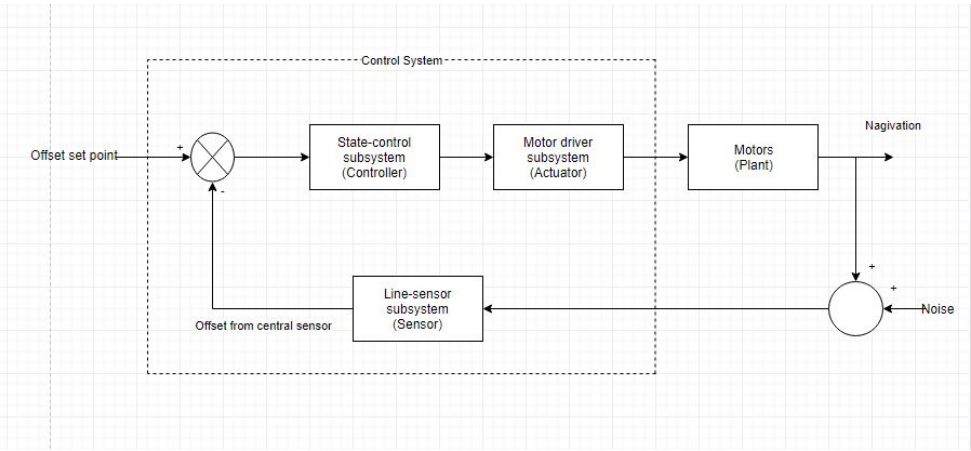

Figure 6.4.1: Functional block diagram of the ARVs control system

6.5 Task 5: Transfer Function

In this section the transfer function will be derived using First order plus time delay (FOPTD).

The general form of for FOPTD can be seen below.

$$\tau \frac{dy(t)}{dt} + y(t) = Kp.u(t - td) \tag{1.5}$$

The general form will then be transformed using the laplace method. The first term is transformed and evaluated at y(0) = 0 as seen bellow.

$$\begin{aligned} \mathscr{L}\left(\tau \frac{dy(t)}{dt}\right) &= \tau(s(Y(s) - y(0)) \\ &= \tau s Y(s) \end{aligned} \tag{1.6}$$

The second term of equation 1.5

$$\mathscr{L}(y(t)) = Y(s) \tag{1.7}$$

Third term of equation 1.5

$$\mathscr{L}(Kp.u(t - td)) = Kp.U(s)e^{-tds} \tag{1.8}$$

The terms can now be combined and evaluated.

$$\begin{aligned} \tau s Y(s) + Y(s) &= Kp.U(s)e^{-tds} \\ Y(s)[\tau s + 1] &= Kp.U(s)e^{-tds} \\ \therefore G(s) = \frac{Y(s)}{U(s)} &= \frac{KP.e^{-tds}}{\tau s + 1} \end{aligned} \tag{1.9}$$

Td is defined as the time delay and τ as the time constant. As seen in section Motor Subsystem Simulations on page 13 the values of TD and τ was determined from the graph as 1s and 1.38s respectively.

$$\therefore G(s) = \frac{Y(s)}{U(s)} = \frac{Kp.e^{-1s}}{1.38s + 1} \tag{1.10}$$

6.6 Task 6: Linearized model

A sensor to shaft distance of $10cm$ and a wheel radius of $2.1cm$ was chosen for the following. The equilibrium conditions were given:

$$\rho_{max} = 33rpm, \therefore u_e = \begin{bmatrix} 3.46 \\ 3.46 \end{bmatrix} \tag{1.11}$$

The system is linearized around the above mentioned equilibrium point. The linearized system will be able to approximate small deviations from the equilibrium point with reasonable precision.

The previously derived matrices from Practical 1 are given bellow:

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & R \times \pi \\ 0 & 0 & 0 \end{bmatrix} \tag{1.12}$$

$$\mathbf{B} = \begin{bmatrix} \frac{R}{2} & \frac{R}{2} \\ 0 & 0 \\ -\frac{R}{L} & \frac{R}{L} \end{bmatrix} \tag{1.13}$$

$$\mathbf{C} = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \tag{1.14}$$

$$\therefore \dot{\mathbf{z}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & R \times \pi \\ 0 & 0 & 0 \end{bmatrix} z(t) + \begin{bmatrix} \frac{R}{2} & \frac{R}{2} \\ 0 & 0 \\ -\frac{R}{L} & \frac{R}{L} \end{bmatrix} u(t) \tag{1.15}$$

$$\therefore \mathbf{y} = \begin{bmatrix} 1 \end{bmatrix} z(t) \tag{1.16}$$

The chosen values can now be substituted resulting in the following linear model:

$$\therefore \dot{\mathbf{z}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 2.1\pi \\ 0 & 0 & 0 \end{bmatrix} z(t) + \begin{bmatrix} 1.05 & 1.05 \\ 0 & 0 \\ -0.21 & 0.21 \end{bmatrix} u(t) \tag{1.17}$$

$$\therefore \mathbf{y} = \begin{bmatrix} 1 \end{bmatrix} z(t) \tag{1.18}$$

6.7 Task 7: Code flow-diagram

The below code flow was used for the serial communication between microprocessors, indicating how the systems respond when new serial input is received:

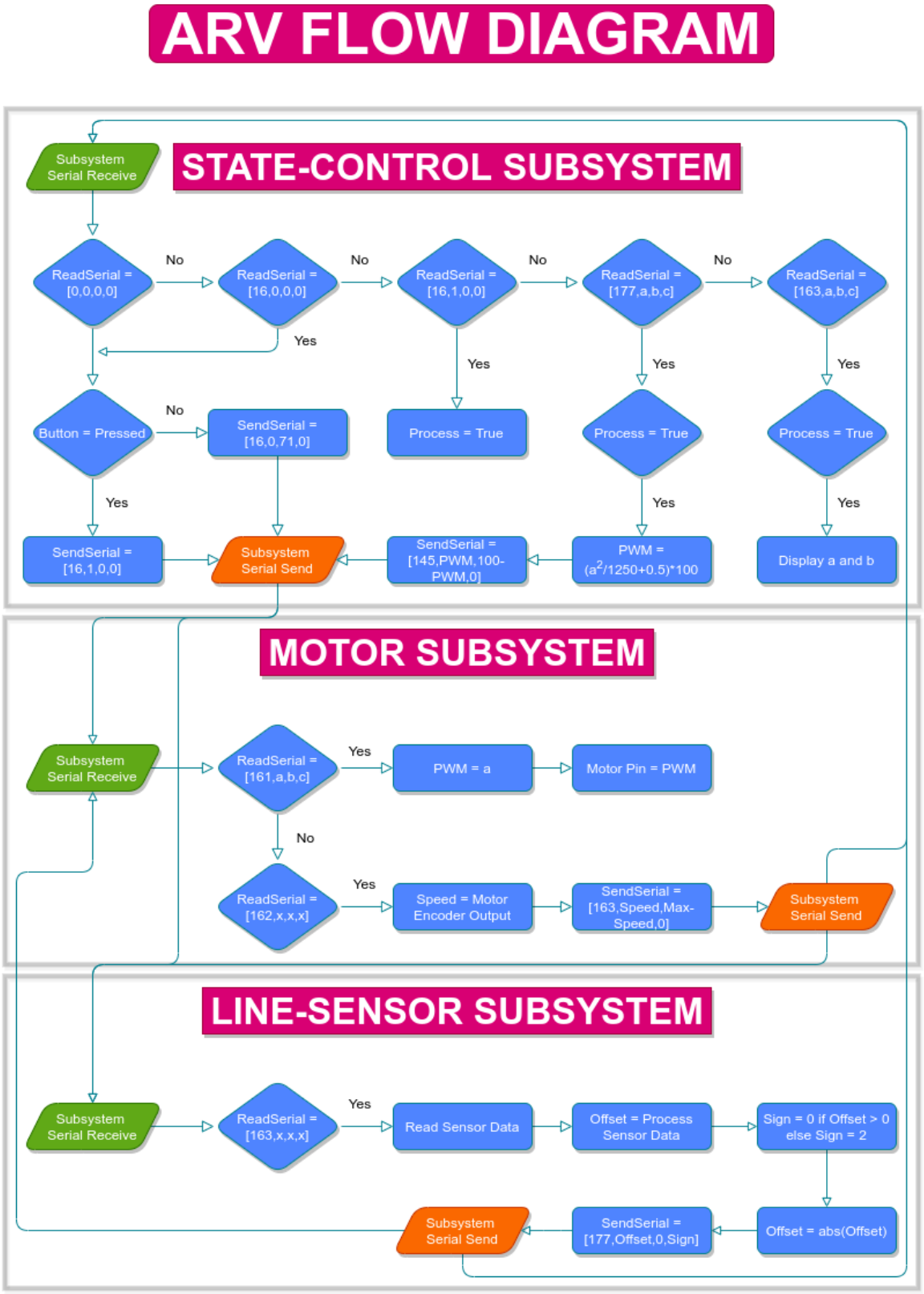

Figure 6.7.1: Code flow-diagram

7 Conclusion

A functional block diagram and flow chart of the system was developed and the transfer function of the system was obtained, the system was also successfully linearised, allowing for a better understanding of how the system operates.

The three individual subsystems (state-control, line-sensor and motor) were designed, simulated and implemented. These subsystems were tested by using a simulation platform, coded using python to simulate the serial data transfer, to ensure that they perform as required. The motor subsystem is designed to allow for control of the motors, allowing the ARV to move when required (it is the actuator of the control system). The subsystem is successfully able to control the operation of the motor as defined by the PWM set point given to it by the state-control subsystem. The line-sensor subsystem is responsible for the data capture process of the ARV. It detects how far the ARV is away from a coloured line and returns this magnitude (and direction) of the displacement to the state-control subsystem. The subsystem is able to quickly and accurately detect colour such that the displacement from the coloured line can be determined. Finally the state control subsystem acts as the controller of the system. It determines when the ARV should turn by analysing the data given to it by the line-sensor subsystem. It then outputs to the motor subsystem to ensure that the ARV drives in the correct direction. The state-control subsystem is able to reliably control the direction and speed of the ARV, given accurate sensor data. Therefore the individual subsystems function as expected and are ready to integrate with one another to form the ARV.

Bibliography

[1] B. Pshenichnyj, "The linearization method," *Optimization*, vol. 18, no. 2, pp. 179–196, 1987.

[2] [Online]. Available: https://meet.google.com/

[3] [Online]. Available: https://www.overleaf.com/p

[4] System, "Arduino uno with rotary encoder to measure rpm," Apr 2014. [Online]. Available: https://forum.arduino.cc/t/arduino-uno-with-rotary-encoder-to-measure-rpm/226182

[5] D. Workshop, "Using rotary encoders with arduino," Sep 2021. [Online]. Available: https://dronebotworkshop.com/rotary-encoders-arduino/

[6] CR7, C. . silver badges66 bronze badges, and C. G. G. . silver badges1212 bronze badges, "Rpm measurement using quadrature encoder," Jun 1966. [Online]. Available: https://arduino.stackexchange.com/questions/49907/rpm-measurement-using-quadrature-encoder

[7] M. B, "Plotting serial port data in real time using python and matplotlib," Feb 2020. [Online]. Available: http://www.mikeburdis.com/wp/notes/plotting-serial-port-data-using-python-and-matplotlib/

[8] S. Fisher, "Circuit diagram web editor," 2013. [Online]. Available: https://www.circuit-diagram.org/editor/

Appendix A

Motor-subsystem Arduino Code

```
#define PWM 5
#define ENCA 2
#define ENCODEROUTPUT 420

volatile long encoderValue = 0;

long previousMillis = 0;
long currentMillis = 0;

int rpm = 10;
boolean measureRpm = false;
int pwm_val = 0;
String Serial_data;
String helper1;
String helper2;
String helper3;
String helper4;



void setup() {
  // put your setup code here, to run once:
  Serial.begin(19200);

  Serial.setTimeout(1);

  pinMode(PWM, OUTPUT);

  pinMode(ENCA, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(ENCA), updateEncoder, RISING);

  previousMillis = millis();
}

void loop() {
  // put your main code here, to run repeatedly:
  while(Serial.available() > 0 ){
```

```
  Serial_data += Serial.readString();

}
int counter = 0;
helper1 = "";
helper2 = "";
helper3 = "";
helper4 = "";
for(int i=0; i < Serial_data.length(); i++){
  if(Serial_data[i] == '[')
    continue;

  else if(Serial_data[i] == ']')
    continue;

  else if(Serial_data[i] == ',')
    counter++;

  else{
    if (counter == 0)
      helper1 += Serial_data[i];
    else if (counter == 1)
      helper2 += Serial_data[i];
    else if (counter == 2)
      helper3 += Serial_data[i];
    else if (counter == 3)
      helper4 += Serial_data[i];
  }
}

if(helper1.toInt() == 161){
  int x = 0;
  float z = 0;
  x = helper2.toInt();
  int temp_pwm = 0;
  z = (float)x/100;
  temp_pwm = z*120;
  pwm_val = round(temp_pwm);
}
currentMillis = millis();
if (currentMillis - previousMillis > 1000) {

  previousMillis = currentMillis;

  rpm = (float)(encoderValue * 60 / ENCODEROUTPUT);
  encoderValue = 0;
}

if(helper1.toInt() == 162){
  String Tosend = "";
  int rpm_overshoot = 0;
  rpm_overshoot = rpm;
  if(rpm_overshoot > 130)
```

```
      rpm_overshoot = 130;
    Tosend = "[163," + String(rpm_overshoot) + ","+ String(130 - rpm_overshoot) + ","
    ↪  + 0 + "]";
    Serial.println(Tosend);
  }
  if(pwm_val > 255)
    pwm_val = 0;

  delay(200);
  analogWrite(PWM, pwm_val);
  Serial_data = "";


}

void updateEncoder()
{
  // Add encoderValue by 1, each time it detects rising signal
  // from hall sensor A
  encoderValue++;
}
```

Appendix B

Motor-subsystem Python Code

```
import serial as sr
import matplotlib.pyplot as plt
import numpy as np
import time

def PlotGraph(x,y,title,xlabel,ylabel,fcount,xlimbot=0,xlimtop=0,
ylimbot=0,ylimtop=0,color = 1):
    plt.figure(fcount)
    plt.plot(x,y,color,linewidth=5)
    plt.xlabel(xlabel,fontsize=30)
    plt.ylabel(ylabel,fontsize=30)
    plt.tick_params(labelsize=20)
    plt.title(title,fontsize=30)


    plt.grid(color = 'green', linestyle = '--', linewidth = 0.5)
    if (xlimbot != 0) or (xlimtop != 0):
        plt.xlim(xlimbot,xlimtop)
    if (ylimbot != 0) or (ylimtop != 0):
        plt.ylim(ylimbot,ylimtop)
        fcount += 1
    plt.show()
    return fcount

arduino = sr.Serial('COM5', 19200, timeout = .1)

Counter = 0
F_count = 1

Time = []
RPM = []
Previousrpm = 0



arduino.close()
arduino.open()
```

```
time.sleep(2)

def Updaterpm():
    arduino.write(bytes("[162,20,90,0]", 'utf-8'))
    temp = arduino.readline().decode('utf-8')
    helper = 0
    toconvert = ""
    for i in range(len(temp)):
        if(temp[i] == ","):
            helper+=1
        elif(helper == 1):
            toconvert += temp[i]
    RecInt = int(toconvert)
    RPM.append(RecInt)
    Time.append(CTime - STime)
    print("Time: " + str(round(CTime - STime,2)) + " RPM: "
     + str(RecInt))

arduino.write(bytes("[161,10,90,0]", 'utf-8'))
TransPacketSend = False
time.sleep(2)

STime = time.time()
CTime = STime

while (CTime - STime) < 30 :
    CTime = time.time()

    if CTime - STime > 0 :
        try:
            Updaterpm()
        except:
            continue

    if (TransPacketSend == False and (CTime - STime) >= 15) :
        arduino.write(bytes("[161,20,80,0]", 'utf-8'))
        try:
            Updaterpm()
        except:
            continue
        TransPacketSend = True

arduino.close()

F_count = PlotGraph(Time, RPM,
"Motor speed over time with step impulse at 15 seconds",
 "Time (s)","Motor speed (rpm)", F_count)

# def write_read(x):
#     arduino.write(bytes(x, 'utf-8'))
#     time.sleep(1)
#     data = arduino.readline()
#     return data
# while True:
```

```
#     num = input("Data packet sent to Arduino: ")
#     value = write_read(num)
#     print("Data packet recieved from Arduino: ",value)
```

Appendix C

Line-sensor subsystem Arduino Code

```
// Author: Timothy Fogwill
// EBB prac 2 code
// Last Update: 06/10/2021

int controlNum = 0;
int recPacket[4] = {0, 0, 0, 0};
bool received = false;
int packet[4] = {0, 0, 0, 0};

int directionOut = 0;
int deviationOut = 0;

// Anything less than 308*1024/5 + 205/1024*5 = 1.5 + 1  = 2.5V
// is considered green track. Anything above that is white
int greenVolLevel = 308;
int voltBuffer = 205;

int threshold = greenVolLevel + voltBuffer;

// will be read using analogRead - ADC values
int sensorData[5] = {0, 0, 0, 0, 0};

void setup() {
  Serial.begin(19200);
}

void loop() {
  delay(0.1);
  // Packet has not been read
  if (Serial.available() > 0) {
    recPacket[controlNum] = Serial.read();

    if (controlNum != 3){
      controlNum++;
      }
    else {
      for (int i = 0; i < 4; i++){
        packet[i] = recPacket[i];
```

```
      }
    controlNum = 0;
    received = true;
    }
  }
else { // Packet has been read
  if ((packet[0] == 163) and (received == true)) {
    received = false;

    // Read voltage values into ADC int vars
    // Top sensor is connected tp A0, botttom to A4
    sensorData[0] = analogRead(A0);
    sensorData[1] = analogRead(A1);
    sensorData[2] = analogRead(A2);
    sensorData[3] = analogRead(A3);
    sensorData[4] = analogRead(A4);

    int dummyVar = 10;
    for (int i = 0; i < 5; i++) {
      if (sensorData[i] < threshold) {
        dummyVar = i;

       // if sensor detects line on more than 1  sensor, favour sensor that causes
       // less turning - less destructive if its wrong. Therefore most readings
       ↪  will
       // be centre or slight turns with sharp turns beig rare (and only used if
       ↪  absolutely needed).

        if ((i == 1) or (i == 3)) {
          break;
          }
        }
      }

    switch(dummyVar) {
      case 0:
        deviationOut = 30;
        directionOut = 2;
        break;
      case 1:
        deviationOut = 15;
        directionOut = 2;
        break;
      case 2:
        deviationOut = 0;
        directionOut = 0;
        break;
      case 3:
        deviationOut = 15;
        directionOut = 0;
        break;
      case 4:
        deviationOut = 30;
        directionOut = 0;
```

```
          break;
        default:
          deviationOut = 0;
          directionOut = 0;
          break;
        }

      sendData(177, deviationOut, 0, directionOut);
      }
    }
}

void sendData(int b1, int b2, int b3, int b4) {
  Serial.write(b1);
  Serial.write(b2);
  Serial.write(b3);
  Serial.write(b4);
  }
```

Appendix D

Line-sensor subsystem Python Code

```
# Author: Timothy Fogwill
# EBB prac 2 logging code
# Last Update: 06/10/2021


import serial
import time
import matplotlib.pyplot as plt

filename = "PositionFie.txt"

def plotPosfile():
    file = open(filename, "r")
    time = []
    pos = []
    Lines = file.readlines()

    for line in Lines:
        hold = int(line.find(',', 0, -1))

        time.append(float(line[0 : hold]))
        pos.append(int(line[hold + 1:]))


    plt.title("Position vs Time of the track relative to the central sensor of the
    ↪  ARV")
    plt.ylabel("Position (mm)")
    plt.xlabel("Time (s)")
    plt.plot(time, pos)
    plt.show()
    file.close()


def testlineSensor():

    ser = serial.Serial(port = "COM5", baudrate = 19200)

    ser.close()
```

```
ser.open()

file = open(filename, "w")

time.sleep(2)

bSent = False
startTime = time.time()
currTime = time.time()

localTime = currTime - startTime

while (localTime < 60):
    currTime = time.time()
    localTime = currTime - startTime


    if (bSent == False):
        time.sleep(0.1)
        ser.write(bytes([163]))
        ser.write(bytes([0]))
        ser.write(bytes([0]))
        ser.write(bytes([0]))
        bSent = True
        print("At " + str(round(localTime, 4)) + "s - Sent: 163, 0, 0, 0")

    elif ((ser.inWaiting() > 0) and (bSent == True)):
        controlNum = 0
        packet = [0, 0, 0, 0]
        bReceived = False


        while (bReceived == False):
            if (ser.inWaiting()):
                hold = ser.read()

                packet[controlNum] = int.from_bytes(hold, 'big') # try little if
                ↪  doesn't work
                controlNum = controlNum + 1

                if (controlNum > 3):
                    breceived = True
                    break

        print("At " + str(round(localTime, 4)) + "s - Received: " +
        ↪  str(packet[0]) + ', ' + str(packet[1]) + ', ' +
             str(packet[2]) + ', ' + str(packet[3]))


        posVal = packet[1]
        if packet[3] == 2:
            posVal = packet[1] * -1

        file.write(str(round(localTime, 4)) + "," + str(posVal) + "\n")
```

```
            bSent = False
            time.sleep(0.1)

    file.close()
    ser.close()
```

Appendix E

State-control subsystem Arduino Code

```
int pos = 0;
int control = -1;
int data1 = -1;
int data2 = -1;
int data3 = -1;

Serial.begin(19200);

void sendSerial(int c, int d1, int d2, int d3) {
  Serial.write(c);
  Serial.write(d1);
  Serial.write(d2);
  Serial.write(d3);
}

bool readSerial() {
  bool newSerial = false;
  while (Serial.available() > 0) {
    int newByte = Serial.read();
    switch (pos) {
      case 0: control = newByte; break;
      case 1: data1 = newByte; break;
      case 2: data2 = newByte; break;
      case 3: data3 = newByte; break;
      default: break;
    }
    pos++;
  }
  if (pos >= 4) {
    newSerial = true;
    pos = 0;
  }
  return newSerial;
}
```

Appendix F

State-control subsystem Python Code

```
import random
import math
import serial

def read():
    return [byte for byte in ser.read(4)]

def send(c, d1, d2, d3):
    p = bytes()
    for byte in [c, d1, d2, d3]:
        p += byte.to_bytes(1, byteorder='big')
    ser.write(p)

ser = serial.Serial('/dev/ttyUSB0', 19200, timeout=1)
send(0, 0, 0, 0)
if read() == [16, 0, 0, 0]:
    send(16, 0, 0, 0)
    print("Connected to subsystem")
else:
    print("Unable to connect")

while(read() != [16, 1, 0, 0]):
    pass
send(16, 1, 0, 0)
print("RACE state detected")

length = 90
sent_offsets = []
read_left_motor_speeds = []
read_right_motor_speeds = []
for i in range(length):
    offset = int(25 * math.sin(math.radians(4 * i)) + random.randint(-1, 1))
    send(177, abs(offset), 0, (0 if offset >= 0 else 2))
    sent_offsets.append(offset)
    c, d1, d2, d3 = read()
    read_right_motor_speeds.append(d1)
    read_left_motor_speeds.append(d2)
for i in range(length):
```

```
    offset = int(i / length * 25)
    send(177, abs(offset), 0, (0 if offset >= 0 else 2))
    sent_offsets.append(offset)
    c, d1, d2, d3 = read()
    read_right_motor_speeds.append(d1)
    read_left_motor_speeds.append(d2)

ser.close()
```